ตารางแฮชแบบกะทัดรัดและทำงานขนานบน GPU

บันทึกสรุปเชิงวิชาการจากบทความ
“Compact Parallel Hash Tables on the GPU”

บันทึกสรุปเชิงวิชาการ (Academic Study Notes)

เอกสารนี้เป็น**บันทึกสรุปเพื่อการศึกษา** ที่เรียบเรียงใหม่ด้วยถ้อยคำและโครงสร้างของผู้จัดทำเอง **มิใช่คำแปล**ของบทความต้นฉบับ ตัวเลขผลการทดลองทั้งหมดอ้างอิงจากต้นฉบับ ส่วนแผนภาพ ตาราง และรหัสเทียมในเอกสารนี้จัดทำขึ้นใหม่เพื่ออธิบาย มิได้คัดลอกจากต้นฉบับ ผู้ที่ต้องการรายละเอียดครบถ้วน (โดยเฉพาะบทพิสูจน์และกราฟผลการทดลอง) ควรอ่านต้นฉบับโดยตรง

บทความต้นฉบับ

Steef Hegeman, Daan Wöltgens, Anton Wijs, and Alfons Laarman. *Compact Parallel Hash Tables on the GPU*. arXiv preprint arXiv:2406.09255 [cs.DS], version 1, 13 June 2024.

`https://arxiv.org/abs/2406.09255`

สังกัดผู้แต่ง: LIACS มหาวิทยาลัยไลเดน, Eindhoven University of Technology และ TNO ประเทศเนเธอร์แลนด์

ซอร์สโค้ด CUDA ของผู้แต่ง:
`https://github.com/system-verification-lab/compact-parallel-hash-tables`

ประเภทเอกสาร: บันทึกสรุปเชิงวิชาการ (ไม่ใช่การแปล)

จัดทำเพื่อประกอบโครงการศึกษา CUDA hash table

บทคัดย่อ

ความเร็วของตารางแฮช (hash table) บน GPU ขึ้นกับจำนวน cache line ที่แต่ละการดำเนินการต้องแตะเป็นหลัก บทความของ Hegeman และคณะถามว่า หากใช้เทคนิค *quotienting* ลดบิตต่อช่อง (slot) จนแต่ละ bucket บรรจุได้มากขึ้น จะได้ทั้งการประหยัดหน่วยความจำและความเร็วหรือไม่ ผู้แต่งเสนอ bucketed cuckoo hashing แบบกะทัดรัด (สำหรับงานสถิต) และ iceberg hashing แบบกะทัดรัดสองระดับที่มี *find-or-put* แบบไร้ล็อก (lockless) พร้อมบทพิสูจน์ความถูกต้อง ผลบน RTX 4090 ชี้ว่าความกะทัดรัดเพิ่ม throughput ราว 10–20% ในกรณีส่วนใหญ่ และมากกว่านั้นเมื่อ bucket มี 32 ช่อง พร้อมลดหน่วยความจำเหลือราวครึ่งหนึ่งหรือเกือบหนึ่งในสี่ บันทึกนี้สรุปแนวคิด ขั้นตอนวิธี การนำไปใช้บน GPU ผลการทดลอง ข้อจำกัด และข้อสังเกตสำหรับโครงการตารางแฮชด้วย CUDA

สารบัญ

1 แรงจูงใจและปัญหาที่ศึกษา **3**

1.1 ปัญหาและช่องว่างของงานวิจัย 3

1.2 ข้อเสนอหลักของบทความ 3

2 ความรู้พื้นฐาน **3**

2.1 ศัพท์และแบบจำลองของตาราง 3

2.2 Cuckoo hashing 3

2.3 Iceberg hashing 3

2.4 Compact hashing ด้วย quotienting 4

3 ตาราง iceberg แบบกะทัดรัดที่ทำงานขนานได้ **4**

3.1 ขอบเขตการออกแบบ 4

3.2 แนวคิดหลัก: “เติมจากช่องแรกที่ว่างเสมอ” 4

3.3 คุณสมบัติที่ผู้แต่งพิสูจน์ 5

4 Cuckoo hashing แบบกะทัดรัด และเหตุที่ไม่มี find-or-put **6**

5 รายละเอียดการนำไปใช้บน GPU **7**

5.1 ข้อพิจารณาด้านสถาปัตยกรรม 7

5.2 การทำงานร่วมกันระดับกลุ่ม (cooperative work sharing) 7

5.3 Permutation ที่ใช้ 7

5.4 ขนาด slot และการจัดวางหน่วยความจำ 7

6 การออกแบบการทดลอง **8**

6.1 สภาพแวดล้อม ขนาดตาราง และความยาวคีย์ 8

6.2 ชุดการทดลอง 8

7 ผลการทดลองที่สำคัญ **8**

7.1 การตีความผล 8

8 ข้อจำกัดและประเด็นที่ควรระวัง **9**

9 สรุป **10**

10 ข้อสังเกตสำหรับการนำไปใช้ในโครงการ CUDA hash table **10**

1 แรงจูงใจและปัญหาที่ศึกษา

1.1 ปัญหาและช่องว่างของงานวิจัย

หน่วยความจำบนการ์ด GPU (global memory) มีจำกัดกว่าของ CPU และการเข้าถึงหน่วยความจำมักเป็นคอขวดหลัก โครงสร้างข้อมูลที่ประหยัดหน่วยความจำจึงอาจได้ทั้งความจุและความเร็ว งานของ Awad และคณะ [3] ชี้ว่าการปรับแต่งตารางแฮชบน GPU โดยแก่นแท้คือการลดจำนวน cache line ที่ต้องแตะต่อการดำเนินการ

ตารางแฮชแบบกะทัดรัด (compact hashing) มีมาตั้งแต่งานของ Cleary [8] และเคยทำให้ขนานบน CPU [14] ด้วยการล็อกแบบหยาบ ซึ่งไม่เหมาะกับ GPU ที่ตารางเร็วที่สุดล้วนเขียนลงตารางโดยตรงด้วยคำสั่ง atomic (ผู้แต่งเรียกว่า *lockless*) และยังไม่เคยมีการวัดผล compact hashing บน GPU มาก่อน

อีกประเด็นคือ*การทำงานแบบพลวัต* (dynamic operation) คือรองรับการค้นหาและการเขียนพร้อมกัน ตาราง bucketed cuckoo ของ [3] เป็นตาราง*สถิต* (static): สร้างครั้งเดียวแล้วค้นหาอย่างเดียว เพราะการแทรกย้ายคีย์ออกจากตารางชั่วคราว การค้นหาพร้อมกันจึงอาจตอบผิดว่า “ไม่พบ” และอาจเกิดสำเนาซ้ำ DyCuckoo [9] ก็ไม่รองรับการแทรกพร้อมกัน ขณะที่งานอย่าง model checking [6, 15] ต้องถามซ้ำ ๆ ว่า “สถานะนี้เคยเห็นหรือยัง ถ้ายังให้บันทึก”

1.2 ข้อเสนอหลักของบทความ

ผู้แต่งเสนอ iceberg แบบกะทัดรัดไร้ล็อกที่มี find-or-put พิสูจน์ได้ว่าถูกต้อง, cuckoo แบบกะทัดรัด (จาก [3]) สำหรับเปรียบเทียบในงานสถิต และโค้ด CUDA แบบเปิดพร้อมการทดลองที่คงจำนวนช่องเท่ากัน ข้อสรุปหลักคือความกะทัดรัดบน GPU ไม่ได้แลกมาด้วยความเร็ว แต่กลับช่วยให้เร็วขึ้น

2 ความรู้พื้นฐาน

2.1 ศัพท์และแบบจำลองของตาราง

ตาราง T แทน*เซต*ของคีย์ (ขยายเป็น map ได้) และประกอบด้วย *bucket* ที่มีหลาย *slot* แต่ละ slot ว่าง (*empty*) หรือเก็บคีย์พร้อมข้อมูลกำกับเล็กน้อย ฟังก์ชันแฮช $h_0, \ldots, h_{H-1}$ กำหนด bucket ที่คีย์อยู่ได้ *อัตราการบรรจุ* (fill factor) คือสัดส่วน slot ที่ถูกใช้; H มากขึ้นบรรจุได้แน่นขึ้นแต่ค้นหาช้าลง ตารางที่คีย์ไม่ย้ายหลังแทรกเรียกว่า*เสถียร* (stable) บทความไม่พิจารณาการลบ ประสิทธิภาพด้านหน่วยความจำขึ้นกับทั้งจำนวนบิตต่อ slot และอัตราการบรรจุที่ทำได้จริง

2.2 Cuckoo hashing

Cuckoo hashing [12] ให้คีย์อยู่ได้เพียง H bucket แต่ยังบรรจุได้แน่น โดยให้คีย์ใหม่ “ไล่” (evict) คีย์เดิม ไปยัง bucket ทางเลือกถัดไปของคีย์นั้น ($h_{(j+1) \bmod H}$) ต่อเป็นทอด ๆ จนพบช่องว่าง หรือสายการไล่ยาวเกิน C จึงถือว่าเต็ม การค้นหาตรวจ $h_0(k), h_1(k), \ldots$ ตามลำดับ และหยุดได้เมื่อพบคีย์หรือพบ bucket ที่ยังไม่เต็ม เพราะคีย์จะไปอยู่ bucket ถัดไปได้ก็ต่อเมื่อถูกไล่จาก bucket ก่อนหน้าเท่านั้น

2.3 Iceberg hashing

Iceberg hashing [7, 13] มีสามระดับ: ระดับ 1 คือ bucket หลัก (primary) ขนาดใหญ่หนึ่งตัวต่อคีย์, ระดับ 2 คือ bucket รอง (secondary) ขนาดเล็กสองตัวต่อคีย์ และระดับ 3 คือรายการโยงนอกตาราง การแทรกลองระดับ 1

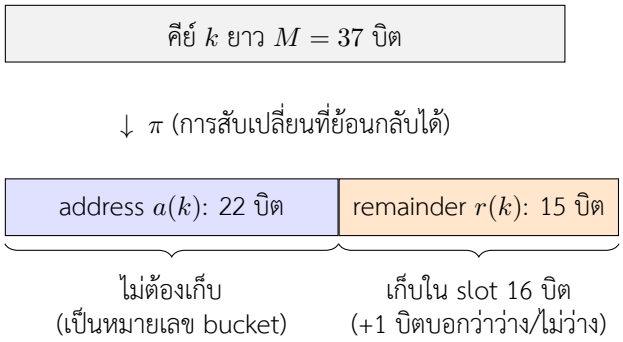
ก่อน ถ้าเต็มเลือก bucket รองที่*ว่างกว่า* (หลัก "power of two choices" [5]) และลงระดับ 3 เมื่อเต็มทั้งหมด การค้นหาตามลำดับเดียวกันและหยุดได้เมื่อพบ bucket ที่ไม่เต็ม ข้อสำคัญคือ iceberg เป็นตาราง*เสถียร* และเมื่อปรับพารามิเตอร์ดี (เช่น IcebergHT ใช้ bucket หลัก 64 ช่อง และระดับ 2 ราว 1/8 ของระดับ 1) การล้นไปยังระดับล่างมีน้อย จึงได้เวลาคงที่ด้วยความน่าจะเป็นสูง

2.4 Compact hashing ด้วย quotienting

แนวคิดของ Cleary [8] คือ *ใช้ตำแหน่งของ bucket เองเป็นข้อมูลส่วนหนึ่งของคีย์* ให้ตารางมี 2^N bucket คีย์เป็นสตริงบิตยาว M บิต และ π เป็นการเรียงสับเปลี่ยน (permutation) บนเซตคีย์

- *address* $a(k)$ = N บิตแรกของ $\pi(k)$ ใช้เลือก bucket
- *remainder* $r(k)$ = $M - N$ บิตที่เหลือของ $\pi(k)$ คือสิ่งที่เก็บจริงใน slot
- กู้คีย์คืนได้จาก $k = \pi^{-1}\big(a(k) \,\|\, r(k)\big)$ โดย $\|$ คือการต่อสตริงบิต

เพราะ π เป็นการสับเปลี่ยน (หนึ่งต่อหนึ่ง) คีย์ต่างกันที่อยู่ bucket เดียวกันย่อมมี remainder ต่างกัน จึงไม่สูญเสียข้อมูล ทุก slot ประหยัดได้ N บิต ซึ่งเป็นลอการิทึมของจำนวน bucket หากใช้หลายฟังก์ชันแฮช (เช่น cuckoo หรือ iceberg) ก็ใช้หลาย permutation π_i ได้ฟังก์ชัน a_i, r_i หลายชุด ข้อสังเกตคือ remainder ที่เก็บใน bucket ทางเลือกที่ i ต้องรู้ว่ามาจาก π_i ใด จึงต้องเก็บดัชนี i ไว้ด้วย (หรือเลือกให้ bucket แต่ละระดับมี permutation ของตนเอง ดังที่ iceberg ทำ)



รูปที่ 1: ตัวอย่าง quotienting ด้วยตัวเลขจากการตั้งค่าการทดลองของบทความ: ระดับหลักมี 2^{27} slot แบ่งเป็น bucket ละ 32 slot จึงมี 2^{22} bucket (แผนภาพจัดทำใหม่โดยผู้สรุป)

3 ตาราง iceberg แบบกะทัดรัดที่ทำงานขนานได้

3.1 ขอบเขตการออกแบบ

ผู้แต่งเลือก iceberg เพราะ cuckoo รองรับการเขียนพร้อมกันได้ยาก (หัวข้อ 4) การดำเนินการหลักคือ FindOrPut(k) ซึ่งคืน found, put หรือ full และมีเพียง**ระดับ 1–2**: T_0 (bucket หลัก B_0 ช่อง, permutation π_0) และ T_1 (bucket รอง B_1 ช่อง, ใช้ π_1, π_2 ร่วมกัน จึงเก็บ $(r_1(k), 0)$ หรือ $(r_2(k), 1)$) คำสั่ง atomic ที่ใช้มีเพียง compare-and-swap CAS$(x, \text{old}, \text{new})$

3.2 แนวคิดหลัก: "เติมจากช่องแรกที่ว่างเสมอ"

หัวใจคือ จะเขียนคีย์ลง slot ใดได้ก็ต่อเมื่อทุก slot ที่มาก่อน (ตามลำดับของคีย์นั้น) *ไม่ว่างและอ่านแล้วว่าไม่ใช่คีย์นี้* สอง thread ที่แทรกคีย์เดียวกันจึงแย่ง slot ว่างแรกช่องเดียวกัน CAS สำเร็จเพียงหนึ่ง อีกตัวอ่านใหม่แล้วตอบ found

ซึ่งใช้ได้เพราะ iceberg เสถียร ค่าที่เขียนแล้วไม่ถูกย้ายอีก

ขั้นตอนวิธีที่ 1 เป็นรหัสเทียมที่ผู้สรุปเรียบเรียงใหม่จากคำอธิบายในต้นฉบับ

ขั้นตอนวิธีที่ 1 Find-or-put ของ iceberg แบบกะทัดรัดสองระดับ (เรียบเรียงใหม่)

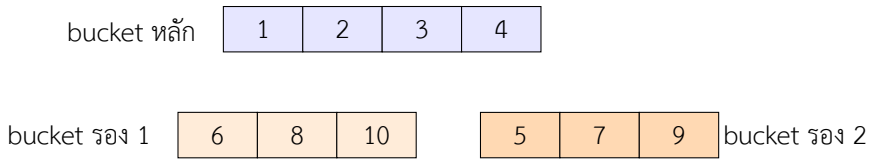
ข้อมูลเข้า: คีย์ k; ตารางร่วม T_0 (B_0 slot/bucket) และ T_1 (B_1 slot/bucket)
ผลลัพธ์: found, put หรือ full

```
1:  function FindOrPut(k)
2:      p ← T0[a0(k)];      v ← r0(k)
3:      loop                                                    ▷ ระดับ 1
4:          snap ← อ่านทุก slot ของ p                           ▷ ภาพถ่าย (snapshot) ภายใน thread
5:          if v ∈ snap then return found
6:          end if
7:          if snap ไม่มี slot ว่าง then break                  ▷ ระดับ 1 เต็ม
8:          end if
9:          i ← ดัชนี slot ว่างแรกใน snap
10:         if CAS(p[i], empty, v) then return put
11:         end if                          ▷ ล้มเหลว: มีผู้อื่นเขียนก่อน วนกลับไปอ่านใหม่
12:     end loop
13:     loop                                                    ▷ ระดับ 2
14:         s1 ← อ่าน T1[a1(k)];      s2 ← อ่าน T1[a2(k)]
15:         if (r1(k), 0) ∈ s1 or (r2(k), 1) ∈ s2 then return found
16:         end if
17:         c ← 1 ถ้า s1 มีของน้อยกว่า s2 อย่างเคร่งครัด มิฉะนั้น c ← 2   ▷ เสมอกันเลือกตัวที่สอง
18:         if s_c ไม่มี slot ว่าง then return full
19:         end if
20:         j ← ดัชนี slot ว่างแรกใน s_c
21:         if CAS(T1[a_c(k)][j], empty, (r_c(k), c − 1)) then return put
22:         end if
23:     end loop
24: end function
```

ข้อสังเกต: เมื่อเลือก bucket รองที่ว่างกว่าแล้วพบว่าเต็ม แปลว่าทั้งสอง bucket เต็ม (เพราะเลือกตัวที่ว่างกว่า) จึงตอบ full ได้ทันที ในตารางสามระดับเต็มรูปแบบ จุดนี้คือจุดที่ควรส่งต่อไประดับ 3

3.3 คุณสมบัติที่ผู้แต่งพิสูจน์

ในภาคผนวก ผู้แต่งกำหนดลำดับดี (well-order) บน slot ทั้ง $B_0 + 2B_1$ ช่องของแต่ละคีย์ให้ตรงกับลำดับที่ขั้นตอนวิธีพยายามเขียน (รูปที่ 2) แล้วพิสูจน์ว่า สำหรับการรัน find-or-put พร้อมกันจำนวนจำกัด (1) ตารางคง*รูปแบบถูกต้อง* คือถ้าคีย์อยู่ใน slot ใด ทุก slot ก่อนหน้าต้องไม่ว่างและไม่ใช่คีย์นั้น จึงไม่มีคีย์ซ้ำ (2) ทุกการเรียกสิ้นสุดและคืน found, put หรือ full (3) put ก็ต่อเมื่อตัวมันเองแทรกคีย์ และ found/put ก็ต่อเมื่อคีย์อยู่ในตารางหลังจบ (4) full หมายถึง bucket ทั้งสามของคีย์เต็มและไม่มีคีย์นั้น ผลคือคีย์เดียวกันจะได้ put อย่างมากหนึ่งครั้ง



รูปที่ 2: ลำดับการลองเขียน (ตัวเลขในช่อง) สำหรับคีย์หนึ่ง เมื่อ $B_0 = 4$ และ $B_1 = 3$ ตามลำดับดี $\prec$ (แผนภาพจัดทำใหม่โดยผู้สรุป)

4 Cuckoo hashing แบบกะทัดรัด และเหตุที่ไม่มี find-or-put

ตาราง cuckoo แบบกะทัดรัดของบทความใกล้เคียงกับ bucketed cuckoo ของ [3] ต่างกันเพียงใช้ permutation แทนฟังก์ชันแฮชและเก็บ remainder แทนคีย์เต็ม จุดที่ต้องเพิ่มคือ เมื่อไล่คีย์ออกจาก slot ต้องกู้คีย์เต็มกลับมาด้วย π_j^{-1} ก่อนจะคำนวณ bucket ถัดไป

ขั้นตอนวิธีที่ 2 การแทรกแบบขนานของ cuckoo แบบกะทัดรัด (เรียบเรียงใหม่)

ข้อมูลเข้า: คีย์ k, จำนวนฟังก์ชัน H, ขีดจำกัดสายการไล่ C

function CuckooPut(k)
  $j \leftarrow 0$
  for $c \leftarrow 1$ **to** C **do**
    $a \leftarrow a_j(k)$; $e \leftarrow (r_j(k), j)$ ▷ สิ่งที่จะเก็บ: remainder + ดัชนี permutation
    $snap \leftarrow$ อ่าน $T[a]$
    if มี slot ว่างใน $snap$ ที่ดัชนี i **then**
      if CAS($T[a][i], empty, e$) **then return** put
      end if ▷ ล้มเหลว: ลองรอบถัดไปด้วยคีย์เดิม
    else
      เลือก i แบบสุ่มเทียม; $(r', j') \leftarrow$ AtomicExchange($T[a][i], e$)
      $k \leftarrow \pi_{j'}^{-1}(a \,\|\, r')$ ▷ กู้คีย์ที่ถูกไล่จาก address + remainder
      $j \leftarrow (j' + 1) \bmod H$ ▷ ไปยัง bucket ทางเลือกถัดไปของคีย์นั้น
    end if
  end for
  return full
end function

การค้นหาตรวจ $r_i(k)$ ใน bucket $a_i(k)$ ตามลำดับ $i = 0, 1, \ldots$ และหยุดเมื่อพบ bucket ที่ไม่เต็มแต่ไม่มีคีย์ (เพราะ bucket ถูกเติมตามลำดับ ถ้า bucket ที่ i ยังไม่เต็ม bucket ถัดไปของคีย์นั้นต้องไม่มีคีย์นั้นอยู่)

เหตุที่ไม่มี find-or-put ที่มีประสิทธิภาพ เพราะ cuckoo ไม่เสถียร การค้นหาอาจพลาดคีย์ที่กำลังถูกไล่อยู่ (แก้ได้ด้วยการซิงโครไนซ์ทั้ง GPU ซึ่งแพงมาก) และแม้แยกเฟสได้ หลาย thread ที่ถือคีย์เดียวกันก็อาจแทรกสำเนาซ้ำหลังคีย์ถูกไล่ออกไป ผู้แต่งจึงใช้ cuckoo กะทัดรัดเฉพาะงานสถิต

5 รายละเอียดการนำไปใช้บน GPU

5.1 ข้อพิจารณาด้านสถาปัตยกรรม

thread ถูกจัดเป็น *warp* ละ 32 ซึ่งควรเลี่ยงการแยกกิ่ง (branch divergence) และควรอ่าน cache line เดียวกันพร้อมกันเพื่อให้การอ่านถูกรวม (coalesced) การเขียนจะเห็นได้โดย thread อื่นเมื่อเขียนด้วย atomic (`atomicCAS`, `atomicExch`) และอ่านแบบ `volatile`

5.2 การทำงานร่วมกันระดับกลุ่ม (cooperative work sharing)

แต่ละ thread ได้รับคีย์ของตนเอง แต่ทั้ง*กลุ่ม* (cooperative group ขนาดเท่ากับ bucket หลัก) ช่วยกันจัดการคีย์ทีละตัว โดยแต่ละ thread อ่านหนึ่ง slot ทั้ง bucket จึงถูกอ่านครั้งเดียวแบบ coalesced แล้วตัดสินใจร่วมกันด้วย primitive ระดับกลุ่ม (shuffle, any, ballot/นับ) ในระดับ 2 ซึ่ง bucket รองมีขนาดครึ่งหนึ่ง กลุ่มแบ่งเป็นสองกลุ่มย่อย (thread คู่/คี่) อ่าน bucket รองคนละตัว แลกค่าภาระเพื่อเลือกตัวที่ว่างกว่า แล้วให้ thread เดียวทำ CAS และกระจายผลด้วย shuffle จุดสำคัญคือ “ช่องว่างแรก” เท่ากับจำนวนช่องที่เต็มพอดี เพราะสมบัติลำดับทำให้ช่องที่เต็มเรียงติดกันจากต้น bucket โครงร่าง CUDA ด้านล่างผู้สรุปเขียนขึ้นเองเพื่อแสดงแนวคิดของระดับ 1 (ไม่ใช่โค้ดของผู้แต่ง; `cas16` เป็นฟังก์ชันสมมติ)

```
// ระดับ 1 ของ find-or-put แบบทั้ง warp ช่วยกัน (โครงร่างเพื่อการศึกษา)
__device__ int level1_fop(volatile uint16_t* T0, uint32_t addr,
                          uint16_t rem, int lane) {
  const unsigned FULL = 0xffffffffu;
  while (true) {
    uint16_t s = T0[addr * 32 + lane];            // coalesced: 1 cache line
    if (__any_sync(FULL, s == rem)) return FOUND;
    unsigned filled = __ballot_sync(FULL, s != EMPTY);
    int load = __popc(filled);                    // ช่องเต็มเรียงติดกันจากต้น
    if (load == 32) return GO_LEVEL2;
    bool ok = false;
    if (lane == load)                             // thread เดียวพยายามเขียน
      ok = cas16(&T0[addr * 32 + load], EMPTY, rem);
    if (__shfl_sync(FULL, ok, load)) return PUT;  // กระจายผลทั้งกลุ่ม
  }
}
```

โค้ดจริงของผู้แต่งยังไม่อ่านช่องที่รู้แล้วว่าเต็มซ้ำ ใช้ค่าที่ `atomicCAS` คืนมาแทนการอ่านใหม่ และรองรับชุดข้อมูลเข้าทุกขนาด

5.3 Permutation ที่ใช้

ผู้แต่งใช้ Feistel รอบเดียวที่มีฟังก์ชันภายในจากตระกูลแฮชเดียวกับ [3] (ผู้ใช้เปลี่ยนเองได้) และพบว่าเวลาคำนวณน้อยจนละเลยได้ โดยทั่วไป Feistel รอบเดียวแบ่งคีย์เป็น (L, R) แล้วให้ $\pi(L, R) = (L \oplus f(R),\ R)$ ซึ่งย้อนกลับได้เสมอไม่ว่า f เป็นอะไร (ตัวอย่างนี้เป็นคำอธิบายของผู้สรุป การแบ่งบิตจริงให้ดูในโค้ดของผู้แต่ง)

5.4 ขนาด slot และการจัดวางหน่วยความจำ

- `atomicCAS` รองรับคำขนาด 16, 32, 64, 128 บิต ส่วน `atomicExch` รองรับ 32, 64, 128 บิต ตาราง iceberg (ใช้แค่ CAS) จึงรองรับ slot 16 บิตได้ ขณะที่ cuckoo (ต้องใช้ exchange ด้วย) เล็กสุดคือ 32 บิต

- slot ที่เล็กกว่านี้ทำได้ด้วยการใช้ atomic ที่ “หยาบกว่า” ข้อมูลจริง แต่น่าจะเสียสมรรถนะ
- ขนาดรวมของ bucket ควรหาร 128 ไบต์ (ขนาด cache line) ลงตัว มิฉะนั้นบาง bucket จะคร่อมสอง cache line
- โค้ดสมมติว่าจำนวน slot ต่อระดับเป็นกำลังของสอง และคีย์ยาว 64 บิต (ปรับเปลี่ยนได้)

6 การออกแบบการทดลอง

6.1 สภาพแวดล้อม ขนาดตาราง และความยาวคีย์

การทดลองหลักใช้ NVIDIA RTX 4090 (24 GB) กับตารางใหญ่ที่สุดที่ยังเหลือที่ให้ข้อมูลเข้าและผลลัพธ์: cuckoo 2^{27} slot และ iceberg ระดับ 1 2^{27} slot กับระดับ 2 2^{24} slot ทุกรุ่นที่เทียบกัน*มีจำนวน slot เท่ากัน* ต่างกันเพียงการแบ่ง bucket และบิตต่อ slot โดยรุ่น 64 บิตเป็นเส้นฐานที่ไม่กะทัดรัด ด้วย bucket หลัก 32 ช่อง ระดับ 1 มี 2^{22} bucket และ slot 16 บิตเก็บ remainder ได้ 15 บิต คีย์จึงยาวได้ $22 + 15 = 37$ บิต ผู้แต่งจึงสุ่มคีย์จาก $[0, 2^{37})$ (รูปที่ 1) ระดับ 2 ประหยัดบิตได้น้อยกว่าและต้องเก็บบิตบอก permutation จึงใช้ slot 32 บิตเสมอ

ตารางที่ 1: สรุปรูปแบบตารางที่ทดลอง (จัดทำใหม่จากคำบรรยายในต้นฉบับ)

ชนิด	slot ต่อ bucket (หลัก)	ความกว้าง slot	หน่วยความจำเทียบเส้นฐาน
Cuckoo	8, 16, 32	64 บิต (เส้นฐาน)	1
Cuckoo	8, 16, 32	32 บิต (กะทัดรัด)	1/2
Iceberg	8, 16, 32 (รอง = ครึ่งหนึ่ง)	64/64 บิต (เส้นฐาน)	1
Iceberg	8, 16, 32	32/32 บิต (กะทัดรัด)	1/2
Iceberg	8, 16, 32	16/32 บิต (กะทัดรัดมาก)	$\approx 9/32$ ตามที่ต้นฉบับระบุ

6.2 ชุดการทดลอง

(1) **Put**: เติมคีย์ไม่ซ้ำจนอัตราการบรรจุ 0.5–0.95; (2) **Find**: ค้นหาคีย์จำนวนครึ่งหนึ่งของ slot โดยแปรสัดส่วนคีย์ที่มีจริง 0–1; (3) **Find-or-put**: คีย์ผสมซ้ำ/ใหม่จำนวนเท่ากับ slot ให้ตารางไปจากอัตรา “ก่อน” ถึง “หลัง” โดยเส้นฐานของ cuckoo คือเรียงด้วย radix sort ของ Thrust, Find แล้วจึง Put; (4) **ข้อมูลจริง** HAVi [15]: ราว $2^{26.1}$ คีย์ 24 บิต (ไม่ซ้ำ $2^{23.9}$) กับ cuckoo 2^{24} slot และ iceberg $2^{24} + 2^{21}$ slot

7 ผลการทดลองที่สำคัญ

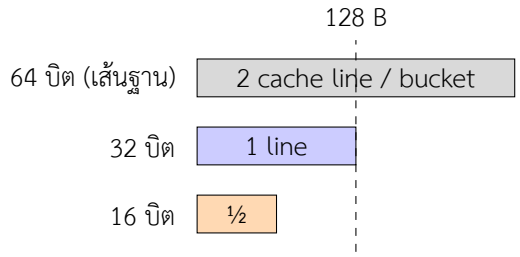
ตารางที่ 2 สรุปตัวเลขหลักที่ผู้แต่งรายงาน (การเพิ่มขึ้นของ throughput เทียบกับรุ่น 64 บิตที่ไม่กะทัดรัด เว้นแต่ระบุเป็นอื่น)

7.1 การตีความผล

ผลชัดที่สุดเมื่อ bucket มี 32 ช่อง เพราะรุ่น 64 บิตใช้ 2 cache line ต่อ bucket แต่รุ่นกะทัดรัดใช้ 1 หรือครึ่ง line (รูปที่ 3) ส่วน cuckoo ค้นหาเร็วกว่าเพราะ iceberg ต้องอ่าน bucket รองทั้งสองเมื่อ bucket หลักเต็มและไม่มีคีย์

ตารางที่ 2: สรุปผลเชิงตัวเลขตามที่รายงานในต้นฉบับ (จัดรูปแบบใหม่โดยผู้สรุป)

การทดลอง	ผลที่รายงาน
Put	รุ่น 32 บิตเร็วขึ้นราว 5–10%; รุ่น slot หลัก 16 บิตเร็วขึ้นราว 15–20%; รุ่น iceberg 32 ช่อง/bucket แบบ 16 บิตเร็วที่สุดในการทดลองนี้ เร็วกว่ารุ่นไม่กะทัดรัดราว 60% อัตราการบรรจุสูงสุด: cuckoo (bucket 16, 32) ถึง 0.95; iceberg สองระดับได้สูงสุด 0.9 (bucket 32)
Find	bucket 32 ช่อง: throughput ราว 2 เท่า; ขนาดอื่นส่วนใหญ่: เพิ่มราว 15–25% cuckoo กะทัดรัดที่ดีที่สุดเร็วกว่า iceberg กะทัดรัดที่ดีที่สุดราว 20–30% ที่อัตราการบรรจุสูง โดยเฉพาะเมื่อคีย์ที่ค้นส่วนใหญ่ไม่มีในตาราง
Find-or-put	ระหว่างรุ่นของ cuckoo (เส้นฐานแบบเรียงลำดับ) ต่างกันน้อย; iceberg 16 บิตเร็วขึ้น 10–20% และ 60–100% เมื่อ bucket หลักมี 32 ช่อง; find-or-put ของ iceberg กะทัดรัดเร็วกว่าเส้นฐาน cuckoo มากกว่า 5 เท่า
ข้อมูลจริง (HAVi)	iceberg กะทัดรัดที่เร็วที่สุดเร็วกว่าเส้นฐาน cuckoo ที่เร็วที่สุดราว 8 เท่า; iceberg กะทัดรัดมากเร็วกว่ารุ่นไม่กะทัดรัดราว 5–15%
Unified memory	ใช้ตารางใหญ่ขึ้นเป็น $2^{29} + 2^{26}$ slot ได้ และรุ่นกะทัดรัดเร็วกว่าราว 10 เท่า (ผู้แต่งเองระบุว่าเป็นการเทียบที่ไม่ยุติธรรม เพราะรุ่นใหญ่ต้องสลับหน้าหน่วยความจำมากกว่า)
GPU อื่น	RTX 3090 และ L40s ให้ผลคล้ายกัน; RTX 2080 Ti ได้ประโยชน์จากความกะทัดรัดน้อยจนละเลยได้



รูปที่ 3: ขนาดของ bucket หลัก 32 ช่องที่ความกว้าง slot ต่าง ๆ เทียบกับ cache line ขนาด 128 ไบต์ (แผนภาพเชิงแนวคิดโดยผู้สรุป)

8 ข้อจำกัดและประเด็นที่ควรระวัง

1. **ไม่มีระดับ 3**: iceberg สองระดับคืน full เมื่อ bucket ทั้งสามเต็ม และบรรจุได้สูงสุดราว 0.9 ซึ่งต่ำกว่า cuckoo (0.95) ผู้แต่งเสนอว่าอาจใช้ slab list ของ [4] เป็นระดับ 3 (และชี้ว่า iceberg ใน [3] ไม่ได้แยกระดับ 1–2 จริงตามทฤษฎีของ [7])
2. **ไม่มีการลบและปรับขนาด**: เหมาะกับงานที่เพิ่มอย่างเดียว การลบอาจทำด้วย tombstone และการปรับขนาดแบบหยุดทั้งระบบ แต่บน GPU มักจองหน่วยความจำทั้งหมดไว้ตั้งแต่ต้นอยู่แล้ว
3. **ความยาวคีย์ผูกกับขนาดตาราง**: ประหยัดได้ $\log_2$(จำนวน bucket) บิต และ slot ต้องเป็นขนาดที่ atomic รองรับ (16–128 บิต)
4. **เส้นฐาน find-or-put ของ cuckoo เรียบง่าย**: ตัวเลข “เร็วกว่า 5–8 เท่า” เทียบกับแนวทางเรียง–ค้น–แทรก ไม่ใช่ cuckoo พลวัตที่ปรับแต่งแล้ว
5. **ขึ้นกับสถาปัตยกรรม**: บน RTX 2080 Ti ประโยชน์แทบไม่มี

6. **ข้อไม่สอดคล้องในต้นฉบับ**: คำบรรยายกราฟ Put กล่าวถึง “10 เท่า” ขัดกับเนื้อความ (5–20%, สูงสุด 60%) และรหัสเทียมแบบกลุ่มมีสัญลักษณ์ไม่ตรงกันเล็กน้อย ควรยึดเนื้อความและโค้ดจริง

9 สรุป

บน GPU การทำ quotienting ทั้งประหยัดหน่วยความจำและเพิ่มความเร็ว โดยเฉพาะเมื่อทำให้ bucket ใช้ cache line น้อยลง iceberg แบบกะทัดรัดสองระดับมีสมรรถนะ Find/Put ใกล้เคียง cuckoo แบบกะทัดรัด และมี find-or-put ไร้ล็อกที่ถูกต้อง ไม่สร้างคีย์ซ้ำ และสิ้นสุดเสมอ แลกกับอัตราการบรรจุสูงสุดที่ต่ำกว่า งานต่อยอดที่ผู้แต่งเสนอคือ การเพิ่มระดับ 3 และการขยายเป็น dictionary คีย์–ค่า ซึ่งความกะทัดรัดน่าจะสำคัญยิ่งขึ้นเพราะคู่คีย์–ค่ากิน cache line มากกว่า

10 ข้อสังเกตสำหรับการนำไปใช้ในโครงการ CUDA hash table

1. **ออกแบบรอบ cache line ก่อนเสมอ**: ให้ขนาด bucket (จำนวนช่อง $\times$ ความกว้าง slot) หาร 128 ไบต์ลงตัว และพิจารณาว่า bucket หนึ่งตัวใช้กี่ line การลดจาก 2 line เหลือ 1 line คือจุดที่ได้ผลมากที่สุด
2. **เลือกสคีมตามรูปแบบงาน**: งานสร้างครั้งเดียวแล้วค้นหาอย่างเดียว (เช่น join, lookup table) ใช้ bucketed cuckoo ได้เต็มที่ งานที่ค้นหาและแทรกปนกัน หรือมีคีย์ซ้ำในชุดข้อมูลเข้า (เช่น dedup, BFS/model checking) ควรใช้สคีมเสถียรอย่าง iceberg ที่มี find-or-put
3. **กติกา “เติมช่องว่างแรกเท่านั้น”** เป็นรูปแบบที่นำไปใช้ซ้ำได้: ถ้าทุก writer เขียนด้วย CAS จาก *empty* ลงช่องว่างแรกตามลำดับเดียวกัน และไม่มีใครแก้ช่องที่ไม่ว่าง จะได้ทั้งการไม่มีคีย์ซ้ำและการค้นหาที่หยุดได้เร็วโดยไม่ต้องใช้ล็อก
4. **ใช้ warp-cooperative probing**: ให้ cooperative group ขนาดเท่ากับ bucket อ่าน bucket แบบ coalesced แล้วใช้ `__ballot_sync`/`__popc` หาจำนวนช่องเต็ม (= ดัชนีช่องว่างแรก) และ `__shfl_sync` กระจายผล
5. **การมองเห็นหน่วยความจำ**: อ่าน slot แบบ `volatile` (หรือ `cuda::atomic_ref` แบบ relaxed) และเขียนด้วย atomic เท่านั้น และนำค่าที่ `atomicCAS` คืนกลับมาใช้แทนการอ่านซ้ำ
6. **วางแผนความยาวคีย์**: คำนวณ บิตที่เก็บได้ $=$ (ความกว้าง slot $-$ บิตสถานะ $-$ บิตดัชนี permutation) $+$ $\log_2$(#bucket) ให้ครอบคลุมความยาวคีย์ที่ต้องการ และระวังว่าระดับที่มี bucket น้อยกว่าจะเก็บคีย์ได้สั้นกว่า
7. **ค่า *empty***: การจองหนึ่งบิตเป็นธงสถานะทำได้ง่าย อีกทางคือห้ามบาง remainder (เช่น 0) แล้วใช้เป็น *empty* ด้วยการเลือก permutation ที่เหมาะสม
8. **Permutation ต้องย้อนกลับได้และถูก**: Feistel รอบเดียวพอสำหรับบทความนี้ แต่ควรทดสอบการกระจายตัวกับข้อมูลจริง (คีย์จาก model checker มักมีโครงสร้าง) และเขียน unit test ว่า $\pi^{-1}(\pi(k)) = k$ สำหรับทุกความกว้างที่รองรับ
9. **เส้นฐานที่ยุติธรรม**: เมื่อวัดผล ให้คงจำนวน slot เท่ากัน เปรียบเทียบรุ่นกะทัดรัดกับรุ่น 64 บิต แยกกรณี positive/negative query และระบุ GPU รุ่นที่ใช้ เพราะผลต่างกันตามสถาปัตยกรรม
10. **สิ่งที่ยังต้องเติมเอง**: ระดับ 3 สำหรับกรณีล้น (หรือกลไกล้นไปยังตารางสำรอง), tombstone สำหรับการลบ, การปรับขนาดแบบหยุดทั้งระบบ และการขยายเป็น map ซึ่งต้องพิจารณาว่าจะเก็บค่าคู่กับ remainder หรือในอาร์เรย์แยก (ใช้ได้เพราะ iceberg เสถียร)

11. **อ้างอิงโค้ดของผู้แต่ง**: ซอร์สโค้ดอยู่ที่ `https://github.com/system-verification-lab/compact-parallel-hash-tables` ก่อนนำส่วนใดมาใช้ควรตรวจสอบสัญญาอนุญาตของรีโพสิทอรีนั้น

เอกสารอ้างอิง

[1] S. Hegeman, D. Wöltgens, A. Wijs, and A. Laarman. Compact Parallel Hash Tables on the GPU. arXiv:2406.09255 [cs.DS], 2024. `https://arxiv.org/abs/2406.09255`

[2] S. Hegeman, D. Wöltgens, A. Wijs, and A. Laarman. Artifact of the paper: Compact Parallel Hash Tables on the GPU. Zenodo, 2024. `https://doi.org/10.5281/zenodo.11638494`

[3] M. A. Awad, S. Ashkiani, S. D. Porumbescu, M. Farach-Colton, and J. D. Owens. Analyzing and Implementing GPU Hash Tables. In *APOCS*, SIAM, 2023.

[4] S. Ashkiani, M. Farach-Colton, and J. D. Owens. A Dynamic Hash Table for the GPU. In *IEEE IPDPS*, pp. 419–429, 2018.

[5] Y. Azar, A. Z. Broder, A. R. Karlin, and E. Upfal. Balanced Allocations. In *STOC*, pp. 593–602, ACM, 1994.

[6] C. Baier and J.-P. Katoen. *Principles of Model Checking*. MIT Press, 2008.

[7] M. A. Bender, A. Conway, M. Farach-Colton, W. Kuszmaul, and G. Tagliavini. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once. arXiv:2109.04548, 2023.

[8] J. G. Cleary. Compact Hash Tables Using Bidirectional Linear Probing. *IEEE Transactions on Computers*, C-33(9):828–834, 1984.

[9] Y. Li, Q. Zhu, Z. Lyu, Z. Huang, and J. Sun. DyCuckoo: Dynamic Hash Tables on GPUs. In *ICDE*, pp. 744–755, 2021.

[10] T. Gunji and E. Goto. Studies on Hashing Part-1: A Comparison of Hashing Algorithms with Key Deletion. *Journal of Information Processing*, 3(1):1–12, 1980.

[11] NVIDIA. *CUDA C++ Programming Guide*, version 12.5, 2024. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`

[12] R. Pagh and F. F. Rodler. Cuckoo Hashing. *Journal of Algorithms*, 51(2):122–144, 2004.

[13] P. Pandey et al. IcebergHT: High Performance Hash Tables Through Stability and Low Associativity. *Proceedings of the ACM on Management of Data*, 1, 2023.

[14] S. van der Vegt and A. Laarman. A Parallel Compact Hash Table. In *MEMICS*, pp. 191–204, Springer, 2012.

[15] A. Wijs and M. Osama. A GPU Tree Database for Many-Core Explicit State Space Exploration. In *TACAS*, pp. 684–703, Springer, 2023.